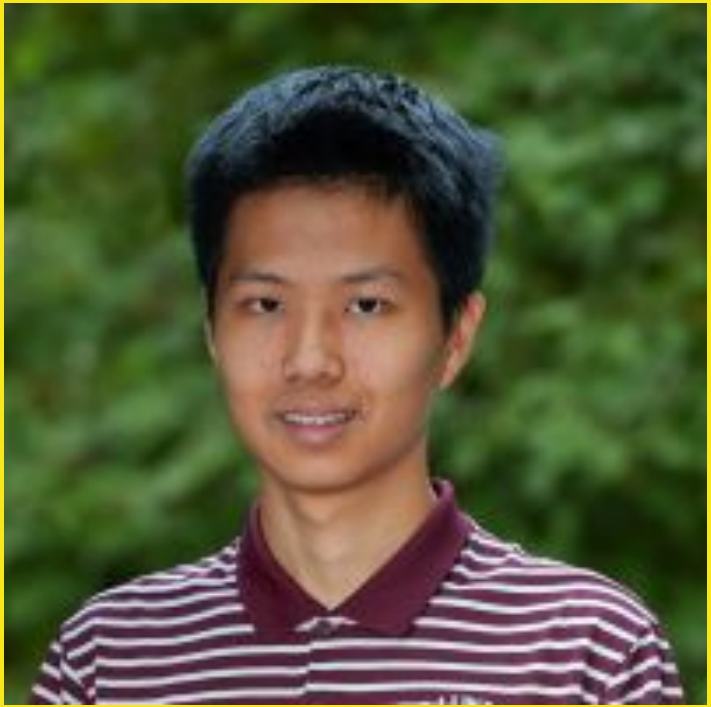
Welcome to the Artificial Intelligence Workshop (Week 3)

Let's Learn How Computers Learn from Information

About Me



Name: Peikang Hu

Current Education: Rising senior at St. Luke's School, New Canaan, Connecticut

Experiences in coding(Python, C++, Java..) and machine learning

Icebreaker

How Many AI Tools Have You Used?

Raise your hand if you've used...

- □ ChatGPT
- □ Siri or Alexa
- □ Google Translate
- □ YouTube recommendations
- □ TikTok or Instagram
- □ Face ID on a phone

Did you know?

You've probably been using AI almost every day!



Workshop Overview

Week 1 – What is Artificial Intelligence (AI)?

Goal: Spark curiosity and understand what AI is.

Week 2 – How Computers Learn

Goal: Introduce linear regression and its application.

Week 3 – A Continuation of Week 2 BUT with more Fun!

Goal: Introduce more advanced AI models.

Week 4 – A Continuation of Week 3 BUT with more Fun!

Goal: Introduce more advanced AI models.

Agenda

- Review the basic concepts of machine learning
- Review the concept of linear regression
- Review the implementation of linear regression
- Learn about the concept of logistic regression
- Learn about the implementation of logistic regression

Review: Machine Learning

What Is Machine Learning?

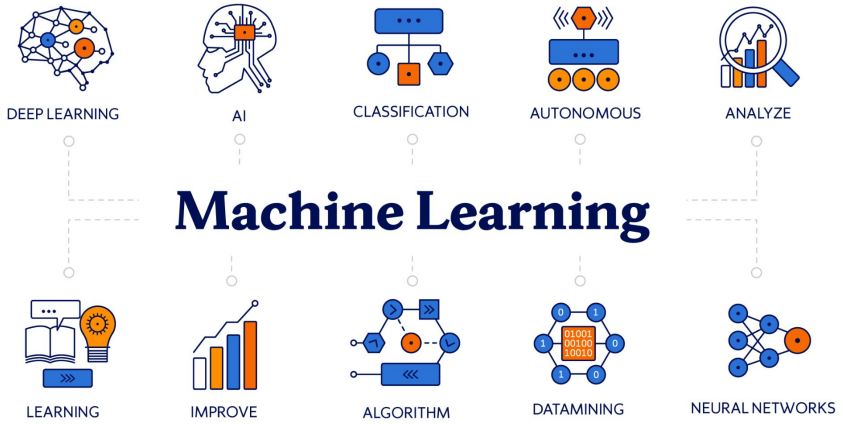
Machine Learning = Teaching computers to learn from examples instead of giving every rule.

Traditional Programming:

Human writes rules
Computer follows rules

Machine Learning:

Give examples
Computer finds patterns



Syracuse University
School of Information Studies

Three Types of Machine Learning

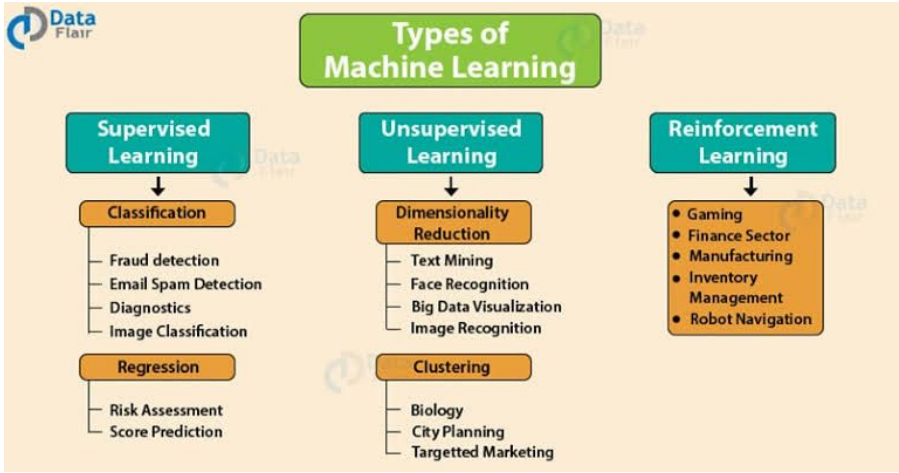
Supervised Learning

Data has **inputs AND correct answers.**

Example: Predicting Exam Scores

Imagine we collect data from several students:

- A student who studied **2 hours** scored **65**.
- A student who studied **4 hours** scored **80**.
- A student who studied **6 hours** scored **92**.
- A student who studied **8 hours** scored **97**.



Using many examples like these, a machine learning model learns the relationship between **hours studied** and **exam score** so it can predict the score for a new student.

Three Types of Machine Learning

Unsupervised Learning

Only inputs.

No labels.

Example:
Group customers based on shopping habits.

Output:
Different customer clusters.



Three Types of Machine Learning

Reinforcement Learning

An agent learns through rewards and penalties.

Examples:

- Chess
- Robotics
- Video games



Two Types of Supervised Learning

Classification

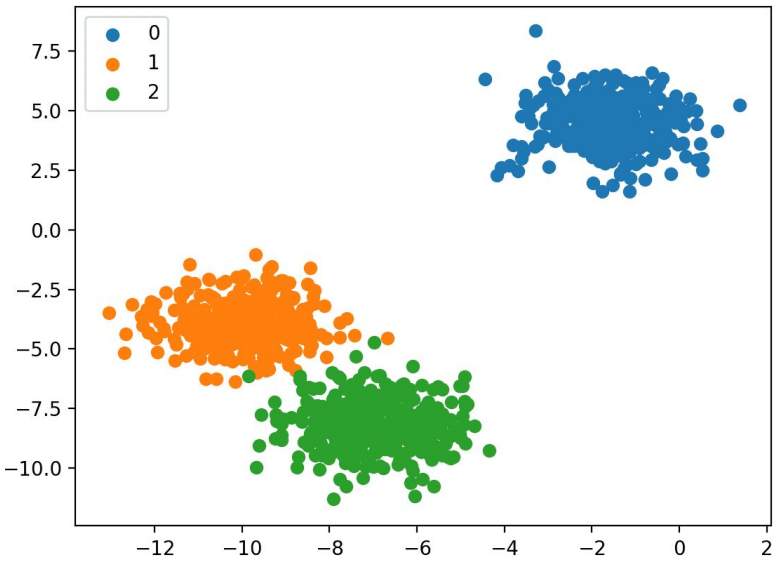
Predict categories.

Examples

- Spam or not spam
- Dog or cat
- Fraud or legitimate

Output:

Discrete labels



Two Types of Supervised Learning

Regression

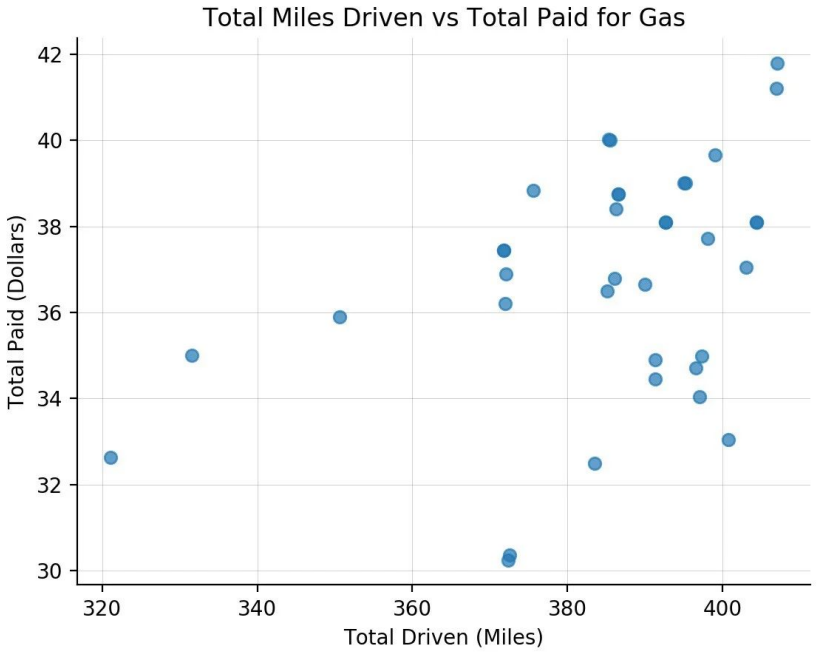
Predict numbers.

Examples

- House prices
- Temperature
- Stock prices
- Salary

Output:

Continuous values



What is Linear Regression?

Linear Regression is the **simplest regression algorithm.**

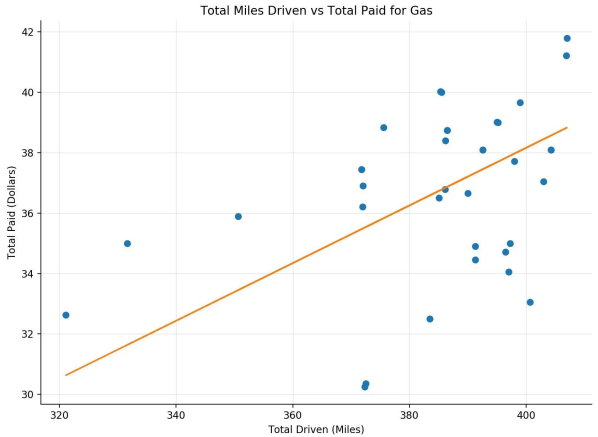
Goal:

Find the best straight line through data.

Example:

Hours Studied → Exam Score

More studying generally leads to higher scores.



The computer finds the line that best describes this relationship.

The Equation of a Line(Machine Learning)

Linear Regression uses almost the same equation:

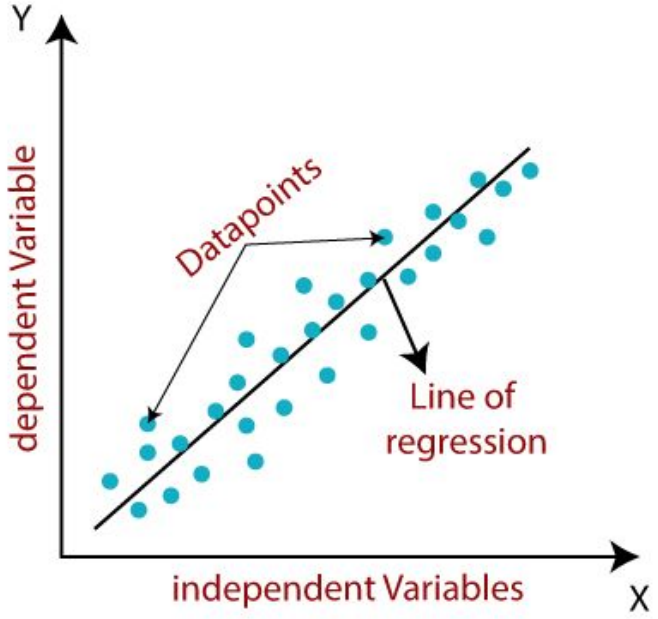
Prediction = Weight × Feature + Bias

or

$\hat{y} = wx + b$

Where:

- **x** = input feature (e.g., hours studied)
- **$\hat{y}$** = predicted value (e.g., predicted exam score)
- **w** = weight (same idea as the slope)
- **b** = bias (same idea as the y-intercept)



Key Idea: Linear Regression is essentially finding the **best possible line** that describes the relationship between the input and the output.

Residuals (Prediction Errors)

Residual = Actual Value − Predicted Value

Example:

- Actual Score = **90**
- Predicted Score = **85**
- **Residual = 90 − 85 = 5**

Key Idea

- A residual measures how far a prediction is from the actual value.
- **Smaller residuals = Better model**

Mean Squared Error (MSE)

Instead of adding errors,

we square them.

Why square?

Positive and negative errors don't cancel.

Large mistakes receive larger penalties.

Goal:

Find the line with the lowest MSE.

$$\text{MSE} = \frac{1}{n}\sum(Y_i - \hat{Y}_i)^2$$

Implementation of Linear Regression

1. Import the necessary libraries

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
```

Explanation: Importing NumPy for numerical operations, Matplotlib for visualization, and Scikit-learn for building the linear regression model.

Implementation of Linear Regression

2. Generating Random Dataset

```
np.random.seed(42)

X = np.random.rand(50, 1) * 100

Y = 3.5 * X + np.random.randn(50, 1) * 20
```

- Sets the **random seed** to 42.
- This makes the random numbers the **same every time** you run the code, so your results are reproducible.

Implementation of Linear Regression

2. Generating Random Dataset

```
np.random.seed(42)

X = np.random.rand(50, 1) * 100

Y = 3.5 * X + np.random.randn(50, 1) * 20
```

- `np.random.rand(50, 1)` generates **50 random numbers** between **0 and 1**.
- `50` = number of data points.
- `1` = one feature (one column).
- Multiplying by `100` scales the values to the range **0 to 100**.
- These are your **input (independent variable)** values.

Implementation of Linear Regression

2. Generating Random Dataset

```
np.random.seed(42)

X = np.random.rand(50, 1) * 100

Y = 3.5 * X + np.random.randn(50, 1) * 20
```

- `3.5 * X` represents the true relationship, meaning that for every 1-unit increase in `X`, `Y` increases by about **3.5**.
- `np.random.randn(50, 1)` generates 50 random values from a normal (Gaussian) distribution with mean **0** and standard deviation **1**.
- **Multiplying by** `20` increases the amount of noise, making the data more realistic instead of perfectly following a straight line.
- The final equation is approximately: `Y = 3.5X + random noise`, where the noise represents natural variation or measurement error.

Implementation of Linear Regression

3. Creating and Training Linear Regression Model

```
model = LinearRegression()
model.fit(X, Y)
```

Explanation: Initialize the model and train it using the generated dataset.

Implementation of Linear Regression

4. Predicting Y Values

```
Y_pred = model.predict(X)
```

Explanation: Using the trained model to predict output values for the input data.

Implementation of Linear Regression

5. Visualizing the Regression Line

```
plt.figure(figsize=(8,6))
plt.scatter(X, Y, color='blue', label='Data Points')
plt.plot(X, Y_pred, color='red', linewidth=2, label='Regression Line')
plt.title('Linear Regression on Random Dataset')
plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.grid(True)
plt.show()
```

Explanation: Plot the original data points along with the best-fit regression line.

Slope and Intercept

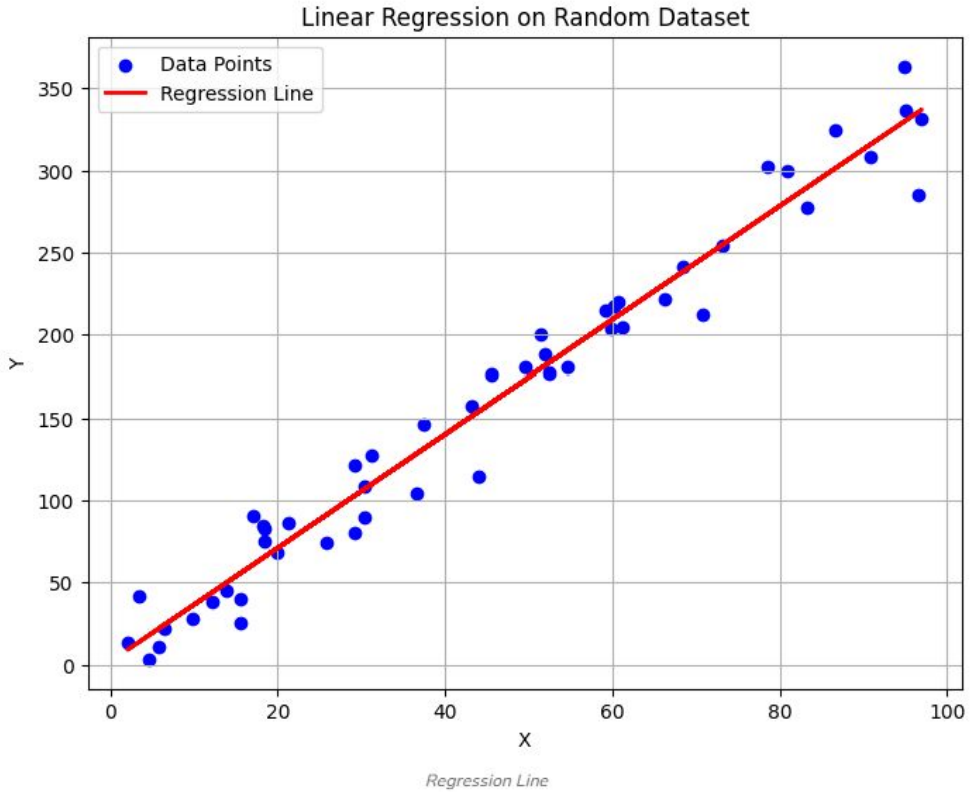
```
print("Slope (Coefficient):", model.coef_[0][0])
print("Intercept:", model.intercept_[0])
```

Explanation: Print the learned slope and intercept values of the regression model.

Output

- The graph shows a **strong positive linear relationship**: as **X increases, Y also increases**.
- Most data points lie **close to the regression line**, indicating the model fits the data well.
- Some points are **above or below the line** due to **random noise (variation)** added to the dataset.
- The **vertical distance** between a data point and the regression line is called the **residual (prediction error)**.
- **Smaller residuals** mean the model's predictions are more accurate.

Output:



Regression Line

Output

```
Slope (Coefficient): 3.4553132007706204
Intercept: 1.9337854893777546
```

Slope (Coefficient): 3.4553

- Here, for every increase of **1** in **X**, **Y increases by about 3.46** on average.
- Since the original data was generated with a true slope of **3.5**, the model's estimate (**3.4553**) is very close.

Intercept: 1.9338

- The intercept is the **predicted value of Y when X = 0**.
- It is close to **0**, which makes sense because the generated data did not include a large constant offset.

Pros and Cons of Linear Regression Models

Pros

- Simple and easy to use
- Fast to train
- Easy to interpret
- Works well for linear data
- Predicts continuous values

Cons

- Only models linear relationships
- Sensitive to outliers
- Can underfit complex data
- Assumes linearity
- Struggles with correlated features

Today's Focus: Logistic Regression

Goal

Predict **which category** something belongs to.

Instead of predicting a number like linear regression, logistic regression predicts a **probability**.

Examples:

- Spam or Not Spam
- Pass or Fail
- Cat or Dog
- Fraud or Not Fraud

Why Can't We Use Linear Regression?

Imagine predicting whether someone passes.

Possible outputs from Linear Regression:

- -0.4 ❌
- 0.2
- 1.8 ❌
- 3.1 ❌

These don't make sense because probabilities must be

- Between 0 and 1

Therefore we need another function.

How does Logistic Regression work?

Logistic Regression is simply

Linear Model

↓

Sigmoid Function

↓

Probability

How does Logistic Regression work?

Step 1: Linear Model

Logistic regression first computes a linear combination of the input features:

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

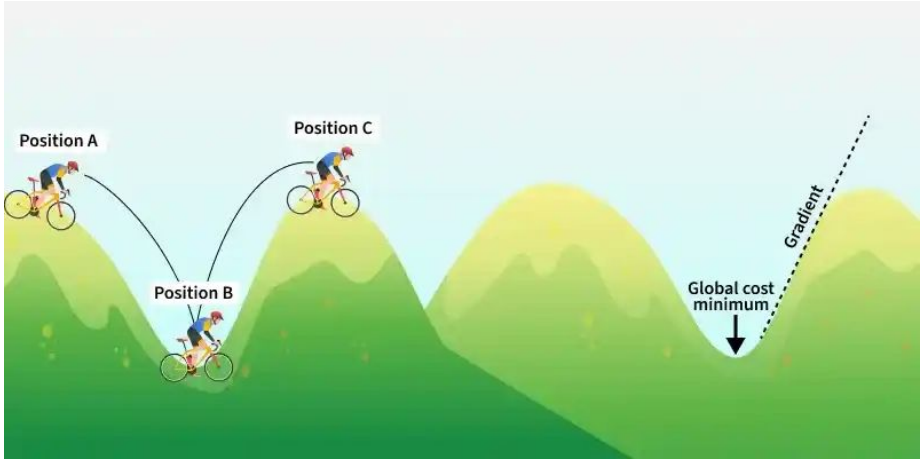
where

- x_i: input features
- w_i: learned weights
- b: bias (intercept)

The value z can be **any real number**, so it is **not yet a probability**.

How does Linear Regression Determine Weights?

- The model starts with **random weights**
- It makes predictions using the input features
- It compares predictions with the actual answers
- A **loss function** measures how wrong the model is
  - Common loss: **Binary Cross-Entropy Loss**
- The model improves using **Gradient Descent**
  - Adjusts weights to reduce errors
  - This process repeats until the model performs well



How does Logistic Regression work?

Step 1: Linear Model (example)

Feature	Input value (x_i)	Learned weight (w_i)
Hours studied	6	0.6
Attendance	90	0.05
Homework completion	80	0.04
Previous test score	85	0.08

The model combines them:

$$z = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b$$

Substitute:

$$z = (0.6)(6) + (0.05)(90) + (0.04)(80) + (0.08)(85) + b$$

Each feature has **one corresponding weight.**

How does Logistic Regression work?

Note: e is a constant and is equal to 2.718

Step 2: The Sigmoid Function

The sigmoid function converts any real number into a probability.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Properties

- Output is always between 0 and 1
- Large positive z → probability near 1
- Large negative z → probability near 0
- z=0 → probability = 0.5

How does Logistic Regression work?

Step 2: The Sigmoid Function (example)

Using our previous example:

The model calculated:

$$z = 3.3$$

Now plug it into sigmoid:

$$P(y = 1) = \frac{1}{1 + e^{-3.3}}$$

Calculate:

$$e^{-3.3} \approx 0.0369$$

Therefore:

$$P(y = 1) = \frac{1}{1 + 0.0369}$$

$$P(y = 1) = 0.964$$

Final Output

$$\boxed{P(\text{Pass}) = 96.4\%}$$

The model predicts:

> "This student has a 96.4% chance of passing."

How does Logistic Regression work?

After applying the sigmoid function, logistic regression gives us a probability.

Probability of passing = 0.964

However, the final answer needs to be a class:

- Pass
- Fail

So we use a decision threshold.

The most common threshold is 0.5:

If probability ≥ 0.5:

→ Predict Class 1 (Positive)

If probability < 0.5:

→ Predict Class 0 (Negative)

The Implementation of Logistic Regression

1. Import the necessary libraries

```
import numpy
from sklearn import linear_model
```

- `numpy` → used for working with arrays (lists of numbers).
- `sklearn.linear_model` → gives us machine learning models, including **Logistic Regression**.

The Implementation of Logistic Regression

2. Generating Random Dataset

```
X = numpy.array([3.78, 2.44, 2.09, 0.14, 1.72, 1.65, 4.92, 4.37, 4.96, 4.52, 3.69, 5.88]).reshape(-1,1)
y = numpy.array([0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1])
```

- X represents the size of a tumor in centimeters.
- y represents whether or not the tumor is cancerous (0 for "No", 1 for "Yes").
- .reshape(-1,1) changes the data into a format that sklearn expects: a 2D vector column with exactly 1 column

The Implementation of Logistic Regression

3. Creating and Training Logistic Regression Model

```
logr = linear_model.LogisticRegression()
logr.fit(X,y)
```

- Creates a logistic regression model.
- The model starts with no knowledge.
- It will learn:
  - a weight (`coefficient`)
  - a bias (`intercept`)

The Implementation of Logistic Regression

4. Predicting Y Values

```
predicted = logr.predict(numpy.array([3.46]).reshape(-1,1))
```

Predict if tumor is cancerous where the size is 3.46mm

```
[0]
[[0.50241666 0.49758334]]
```

The Implementation of Logistic Regression

5: Obtain Probability Values

```
def logit2prob(logr,x):
  log_odds = logr.coef_ * x + logr.intercept_
  odds = numpy.exp(log_odds)
  probability = odds / (1 + odds)
  return(probability)
```

- Converts the model's output into a **probability (0–1)** for class 1 (cancerous).
- Calculates **log odds** using the learned weight and bias: `weight × input + bias`.
- Uses `exp()` to convert log odds into **odds**.
- Uses the sigmoid formula to turn odds into a **probability percentage**.
- Returns the final prediction confidence.

The Implementation of Logistic Regression

```
[[0.60749955]
 [0.19268876]
 [0.12775886]
 [0.00955221]
 [0.08038616]
 [0.07345637]
 [0.88362743]
 [0.77901378]
 [0.88924409]
 [0.81293497]
 [0.57719129]
 [0.96664243]]
```

- The probability that a tumor with the size 3.78mm is cancerous is 61%.
- The probability that a tumor with the size 2.44mm is cancerous is 19%.
- The probability that a tumor with the size 2.09mm is cancerous is 13%.

The Pros and Cons of Logistic Regression

Pros of Logistic Regression

- ✅ Simple and easy to understand
- ✅ Fast to train and predict
- ✅ Works well with small datasets
- ✅ Outputs probabilities, not just classifications
- ✅ Good baseline model for binary classification

Cons of Logistic Regression

- ❌ Assumes a linear relationship between features and the outcome
- ❌ Struggles with complex patterns
- ❌ Needs feature scaling for some datasets
- ❌ Sensitive to outliers
- ❌ Usually less accurate than advanced models for complicated problems

A Preview of Next Week: Neural Networks

You have already learned:

Linear Regression

Find the best line to predict a number

Logistic Regression

Use a weighted combination of features to make a decision

But what if we could:

- Combine **many logistic/linear regression models**
- Let them learn different patterns
- Build layers of increasingly complex decisions


Neural Networks!!!


COME TO THE AI
WORKSHOP NEXT WEEK
on August 15!!!!!!!!